Специальность: 5-04-0612-02 «Разработка и сопровождение программного обеспечения информационных систем»	Дисциплина: «Основы кроссплатформенного программирования»

ЛАБОРАТОРНАЯ РАБОТА № 4
Инструкционно-технологическая карта

Тема: Разработка, отладка и испытание программ с использованием системы сборки проекта Maven

Цель: Научить собирать проекты с помощью maven, компилировать и запускать проекты в командной строке.

Время выполнения: 2 часа

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучить теоретические сведения;
2. Ответить на контрольные вопросы;
3. Откомпилировать примеры программ из раздела «Теоретические сведения»;
4. Выполнить ИДЗ.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ДЛЯ ВЫПОЛНЕНИЯ РАБОТЫ

Gradle – один из нескольких инструментов разработки Java, представленных во всеобщем руководстве разработчика Java от Stackify, но это не единственный инструмент автоматизации сборки, который следует рассмотреть. Maven – более старая и часто используемая альтернатива, но какая система сборки лучше всего подходит для вашего проекта? Поскольку другие инструменты, такие как Spring, позволяют разработчикам выбирать между этими двумя системами, в сочетании с растущим числом интеграций для обеих, решение в значительной степени зависит от вас.

Размер вашего проекта, потребность в кастомизации и некоторые другие переменные могут помочь вам сделать выбор. Давайте посмотрим. Что такое Gradle?

Gradle – это система автоматизации сборки, которая является полностью открытым исходным кодом и использует концепции, которые вы видите в Apache Maven и Apache Ant. Она использует специфический язык, основанный на языке программирования Groovy, что отличает ее от Apache Maven, который использует XML для конфигурации проекта. Она также определяет порядок выполнения задач с помощью направленного ациклического графа.

Несколько разработчиков создали Gradle и впервые выпустили его в 2007 году, а в 2013 году он был принят компанией Google в качестве системы сборки для проектов Android. Она была разработана для поддержки

многопроектных больших сборок. Она также позволяет дополнять вашу сборку, поскольку знает, какие части вашего проекта обновляются. Задачи, которые зависят от обновленных частей повторно не выполняются. На данный момент последним стабильным релизом является версия 3.4, которая была выпущена в феврале 2017 года. Она поддерживает разработку и последующее развертывание с использованием Java, Scala и Groovy, а в будущем будут представлены другие рабочие процессы и языки.

Что такое Maven?

Maven используется для автоматизации сборки проектов с использованием Java. Он помогает составить схему сборки конкретного программного обеспечения, а также его различных зависимостей. Он использует XML-файл для описания проекта, который вы собираете, зависимостей программного обеспечения от сторонних модулей и частей, порядка сборки, а также необходимых плагинов. Существуют заранее определенные цели для таких задач, как упаковка и компиляция.

Maven загружает библиотеки и плагины из различных репозиториев, а затем помещает их все в кэш на вашей локальной машине. Хотя Maven преимущественно используется для проектов на Java, вы можете использовать его для Scala, Ruby и C#, а также для множества других языков.

Gradle в сравнении с Maven

Существуют некоторые фундаментальные различия в подходе этих двух систем к сборке. Gradle основан на графе зависимостей задач, в котором задачи – это то, что выполняет работу, в то время как Maven основан на фиксированной и линейной модели фаз. В Maven цели привязываются к фазам проекта, и цели выполняют ту же функцию, что и задачи в Gradle, являясь "элементами, которые выполняют работу".

С точки зрения производительности, обе программы позволяют параллельно выполнять многомодульные сборки. Однако Gradle позволяет выполнять инкрементные сборки, поскольку проверяет, какие задачи обновлены, а какие нет. Если это так, то задача не выполняется, что дает вам гораздо меньшее время сборки. Другие отличительные особенности производительности, которые можно найти в Gradle, включают:

Инкрементная компиляция для классов Java

Избегание компиляции для Java

Использование API для инкрементальных подзадач

Демон компилятора, который также значительно ускоряет компиляцию. Что касается управления зависимостями, то и Gradle, и Maven могут работать с динамическими и переходными зависимостями, использовать сторонние кэши зависимостей и читать формат метаданных POM. Вы также можете объявлять

версии библиотек через центральное определение версий и обеспечивать централизованное версионирование. Оба продукта загружают переходные зависимости из своих репозиториев артефактов. Maven имеет Maven Central, а Gradle – JCenter, кроме того, вы можете определить свой

собственный репозиторий компании. Если требуется несколько зависимостей, Maven может загружать их одновременно.

Gradle, однако, выигрывает, когда речь идет о зависимостях API и реализации, а также о возможности одновременного безопасного кэширования. Он также хранит метаданные репозитория вместе с кэшированными зависимостями, гарантируя, что два или более проектов, использующих один и тот же кэш, не перезапишут друг друга, а также имеет кэш на основе контрольной суммы и может синхронизировать кэш с репозиторием. Кроме того, Gradle совместим с IVY Metadata, что позволяет определять пользовательские правила для указания версии для динамической зависимости и разрешать конфликты версий. Эти возможности недоступны в Maven.

Ниже приведены другие возможности управления зависимостями, которые вы можете найти только в Gradle:

Использование правил подстановки для совместимых библиотек

Использование правил ReplacedBy

Более качественное управление метаданными

Возможность динамической замены проектных зависимостей на внешние и наоборот.

Gradle также облегчает работу с составными сборками, позволяя работать со специальными и постоянными составными сборками, а также объединять различные сборки и импортировать составную сборку в Eclipse или IntelliJ IDEA.

Что касается моделей выполнения, то обе имеют группы задач и описания. Обе позволяют собирать только указанный проект и его зависимости. Gradle, однако, имеет полностью настраиваемую DAG, в то время как в Maven цель может быть привязана только к одной другой цели. Несколько целей имеют форму упорядоченного списка. Gradle также позволяет исключать задачи, делать переходные исключения и определять зависимости между задачами. По сравнению с другими решениями Gradle также имеет расширенные возможности для упорядочивания задач и финализаторов.

Администрирование инфраструктуры сборки – еще одна сильная сторона Gradle, поскольку он использует обертки, позволяющие автоматическую инициализацию, в то время как для Maven необходимо расширение для поддержки самостоятельной инициализации сборок. Gradle также позволяет настраивать среды сборки на основе версий без необходимости настраивать их вручную. Он также допускает создание пользовательских дистрибутивов.

В целом, оба инструмента имеют свои сильные и слабые стороны. Индивидуальные сборки. С помощью Maven вы можете легко определить метаданные и зависимости вашего проекта, но создание очень индивидуальной сборки может стать кошмаром для пользователей Maven.

Файл POM может легко раздуться по мере роста проекта и впоследствии превратиться в нечитаемый XML-файл.

Управление зависимостями и структура каталогов. Тем не менее, Maven обеспечивает простое, но эффективное управление зависимостями, а поскольку он имеет структуру каталогов для ваших проектов, у вас есть своего рода стандартная схема для всех ваших проектов. Он использует декларативный XML-файл для своего POM-файла и имеет множество плагинов, которые вы можете использовать. Gradle использует структуру каталогов, которую вы видите в Maven, но она может быть настроена. Он также использует тот же формат GAV, который Maven использует для идентификации артефактов.

Плагины и интеграции. Maven также поддерживает широкий спектр этапов жизненного цикла сборки и легко интегрируется со сторонними инструментами, такими как CI-серверы, плагины покрытия кода, системы репозиториев артефактов и др. Что касается плагинов, то в настоящее время количество доступных плагинов растет, и есть крупные поставщики, у которых есть плагины, совместимые с Gradle. Тем не менее, количество доступных плагинов для Maven все еще больше, чем для Gradle.

Гибкость. Gradle, с другой стороны, очень гибкий и основан на скрипте. Пользовательские сборки было бы легко делать на Gradle. Однако, поскольку Gradle практически недавно появился, количество разработчиков, знающих Gradle изнутри, может быть ограничено.

Создание проекта в IntellJ IDEA

1. Запустите среду IDE IntelliJ IDEA.

2. В среде IDE выберите «File» –> «New» –> «Project»

3. В «Generators» выберите «Maven Archetype» и выполните следующие действия:

• введите *название проекта* в поле «Name»;

• выберите каталог для хранения проекта в поле «Location» (по умолчанию C:\Users\User\IdeaProjects)

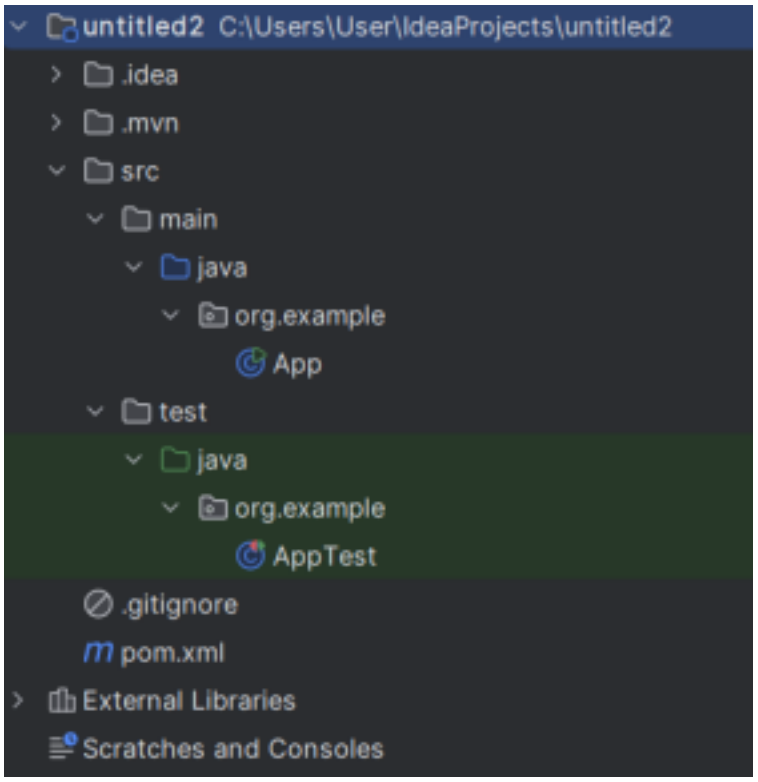
• в поле «JDK» выбираем версию или скачиваем

• «Catalog» выбираем «Maven Central» (если есть подключение к интернету) или «Internal» (если нет подключения к интернету) • «Archetype» выбираем «org.apache.maven.archetypes:maven archetype-quickstart»

4. Нажмите кнопку «Create»

```
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------
[INFO] Total time:  22.250 s
[INFO] Finished at: 2026-08-25T15:12:36+03:00
[INFO] ------------------------------------------

Process finished with exit code 0
```

Maven – инструмент для управления и сборки проектов

Команды:

clean – удаляются все скомпилированные файлы из каталога target (место, в котором сохраняются готовые артефакты);

validate – идет проверка, вся ли информация доступна для сборки проекта;

compile – компилируются файлы с исходным кодом;

test – запускаются тесты;

package – упаковываются скомпилированные файлы (в jar, war и т.д. архив);

verify – выполняются проверки для подтверждения готовности упакованного файла;

install – пакет помещается в локальный репозиторий. Теперь он может использоваться другими проектами как внешняя библиотека; site – создается документация проекта;

deploy – собранный архив копируется в удаленный репозиторий.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое Gradle
2. Что такое Maven
3. В чем их отличие
4. Команды для работы

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

6

Сгенерировать проект, используя maven. Фразу “Hello, world” заменить на “Hello, ИМЯ”. Скомпилировать, запустить.

ДОМАШНЕЕ ЗАДАНИЕ

Индивидуальное задание

ЛИТЕРАТУРА

Блинов, И. Н., Романчик, В. С. Java from EPAM : учеб.-метод. пособие / И. Н. Блинов, В. С. Романчик. Минск: Четыре четверти, 2021. 560 с. .

Преподаватель А.С.Кибисова